

Angelina Podojako s1125886

Report on the solutions of all the exercises. Briefly describe the choices you have made in your models and the LTL properties that you checked.

Add a short reflection: what was the division of tasks in your group? **-I am the only one in group, everything was written by myself.**

ANSWER

1) 55pts

I used semaphore file, which was changed according to exercise.

Firstly, I wrote logics for n=2, after added some details for n=3 (like check not only second but 2 flags in l3,l5,l7,l9,l11).

I added array for flags, added states and their logic as well as flags itself.

FOR n=2

```
MODULE main
VAR
  flag : array 0..1 of 0..4; --for processes 1 and 2 are flags 0 to 4
  proc0 : process Process(0, flag[0], flag[1]); --id, itsflag, otherflag
  proc1 : process Process(1, flag[1], flag[0]);
ASSIGN
  init(flag[0]) := 0;
  init(flag[1]) := 0;

-- ex 2 mutual exclusion
LTLSPEC
  G !(proc0.state = l10 & proc1.state = l10)

-- ex 2 each process that wants to enter its critical section will eventually succeed in doing so.
LTLSPEC
  G (proc0.state = l2 -> F proc0.state = l10)
LTLSPEC
  G (proc1.state = l2 -> F proc1.state = l10)

MODULE Process(id, my_flag, other_flag)
VAR
  state : {l0, l1, l2, l3, l4, l5, l6, l7, l8, l9, l10, l11, l12};
ASSIGN
  init(state) := l0;

  next(state) :=
    case
      state = l0 : l1;
      state = l1 : l2;    --l1: = NC
      state = l2 : l3;    --l2: flag[i] = 1
      state = l3 & (my_flag < 3) & (other_flag < 3) : l4;  --to get in l4: in l3 wait until all flag<3
      state = l3 : l3;    --if not satisfied condition -- wait in l3
      state = l4 : l5;    -- l4: flag[i] := 3
      state = l5 & (other_flag = 1) : l6; --if anyone flag=1
      state = l5 & !(other_flag = 1) : l8; --otherwise go to l8
      state = l6 : l7;    -- l6: flag[i] := 2
      state = l7 & (other_flag = 4) : l8; --l7: wait until someone flag=4
      state = l7 : l7;    --otherwise
      state = l8 : l9;    -- l8: flag[i] := 4
      state = l9 & (id = 0 ? TRUE : other_flag < 2) : l10; --wait until left flag<2 (id=0 = no left and all time true)
      state = l9 : l9;
      state = l10 : l11; --CS
      state = l11 & (id = 1 ? TRUE : (other_flag < 2 | other_flag > 3)) : l12; --wait until right flag<2 and >3 (for id=1 true since no right
    end case
  proc)
  state = l11 : l11;
```

Angelina Podojako s1125886

```
state = l12 : l0;    -- l12: flag[i] := 0
TRUE : state;
esac;
```

```
next(my_flag) :=
case
state = l2 : 1; -- flag[i] := 1
state = l4 : 3; -- flag[i] := 3
state = l6 : 2; -- flag[i] := 2
state = l8 : 4; -- flag[i] := 4
state = l12 : 0; -- flag[i] := 0
TRUE : my_flag;
esac;
```

FAIRNESS

Running

FOR n=3

MODULE main

VAR

```
flag : array 0..1 of 0..4; --for processes 1 and 2 are flags 0 to 4
proc0 : process Process(0, flag[0], flag[1], flag[2]); --id, itsflag, otherflag
proc1 : process Process(1, flag[1], flag[0], flag[2]);
proc2 : process Process(2, flag[2], flag[0], flag[1]);
```

ASSIGN

```
init(flag[0]) := 0;
init(flag[1]) := 0;
init(flag[2]) := 0;
```

-- ex 2 mutual exclusion

LTLSPEC

```
G !((proc0.state = l10 & proc1.state = l10) |
  (proc0.state = l10 & proc2.state = l10) |
  (proc1.state = l10 & proc2.state = l10))
```

-- ex 2 each process that wants to enter its critical section will eventually succeed in doing so.

LTLSPEC

```
G (proc0.state = l2 -> F proc0.state = l10)
```

LTLSPEC

```
G (proc1.state = l2 -> F proc1.state = l10)
```

LTLSPEC

```
G (proc2.state = l2 -> F proc2.state = l10)
```

MODULE Process(id, my_flag, left_flag, right_flag)

VAR

```
state : {l0, l1, l2, l3, l4, l5, l6, l7, l8, l9, l10, l11, l12};
```

ASSIGN

```
init(state) := l0;
```

```
next(state) :=
```

```
case
```

```
state = l0 : l1;
state = l1 : l2;    --l1: = NC
state = l2 : l3;    --l2: flag[i] = 1
```

```
state = l3 & (my_flag < 3) & (left_flag < 3) & (right_flag < 3) : l4;    --to get in l4: in l3 wait until all flag<3
```

```
state = l3 : l3;    --if not satisfied condition -- wait in l3
```

```
state = l4 : l5;    -- l4: flag[i] := 3
```

```
state = l5 & ((left_flag = 1) | (right_flag = 1)) : l6; --if anyone flag=1
```

```
state = l5 & !((left_flag = 1) | (right_flag = 1)) : l8; --otherwise go to l8
```

```
state = l6 : l7;    -- l6: flag[i] := 2
```

```
state = l7 & ((left_flag = 4) | (right_flag = 4)) : l8; --l7: wait until someone flag=4
state = l7 : l7; --otherwise

state = l8 : l9;    -- l8: flag[i] := 4
state = l9 & (id = 0 ? TRUE :
  id = 1 ? (left_flag < 2) :
  id = 2 ? (left_flag < 2) : TRUE) : l10; --wait until left flag<2 (id=0 = no left and all time true)
state = l9 : l9;

state = l10 : l11; --CS
state = l11 & (id = 0 ? (right_flag < 2 | right_flag > 3) :
  id = 1 ? (right_flag < 2 | right_flag > 3) :
  id = 2 ? TRUE : TRUE) : l12; --wait until right flag<2 and >3 (for id=1 true since no right proc)
state = l11 : l11;

state = l12 : l0;    -- l12: flag[i] := 0
TRUE : state;
esac;

next(my_flag) :=
case
state = l2 : 1; -- flag[i] := 1
state = l4 : 3; -- flag[i] := 3
state = l6 : 2; -- flag[i] := 2
state = l8 : 4; -- flag[i] := 4
state = l12 : 0; -- flag[i] := 0
TRUE : my_flag;
esac;

FAIRNESS
running
```

2) 20pts

I just rewrote what has been asked in LTL.

- 1) check that it never happens that both processes are in the critical section at the same time
- 2) check that if process x wants to enter the critical section, it will eventually get there.

I got everywhere true for n=2 and n=3.

3) 25pts

Added reading index in both n=2 and n=3. This I did to make checks in l3 and l7 non atomic. So, only one flag is read per transition and not all at once.

In real life scenarios each flag is read by one, not all at once.

I am not sure about code for n=3, since have not time to work on it properly. But I managed to finish with n=2 (had a lot of errors before)

4) +20bonus_pts

Did the assignment give you additional insight? What was your experience with NuSMV?

Angelina Podojako s1125886

report
all your NuSMV models and LTL properties

ANSWER

It is my first experience with NuSMV at all. I had used different logics thought in the first course of Formal Logic course (AI bachelor 1st year). But we never wirked with any environment like this, and never used LTL to check anything. It was very interesting.

The pre-assginment paper helped a lot to get familiar with NuSMV.